

Performance

The best and certainly the most rewarding of all best practices listed here. It should perhaps be mandatory to follow certain practices when designing an angular application (especially one that is one day expected to grow). For example, It is vital to know beforehand whether you have a component that only expects immutable data to be passed in via a property, since we can then tweak the `ChangeDetectionStrategy` of the component tree and allow for much faster DOM refreshes.

This can have a tremendous impact on the performance of large applications where typically a change in DOM is cascaded to thousands of components, so that each spend a couple of milliseconds reacting to it.

Server side (universal) rendering

Universal means rendering pages on both the server and client side. This improves the initial load time of applications that do not have to wait for the JavaScript to complete loading, instead the server does the rendering and sends a response of the rendered page that angular 2 directly displays on the page. Such a technique usually implies the use of frontend JavaScript and Node.js because they allow for the re-usage of libraries, allowing browser JavaScript code to be run in the Node.js environment with very little modification. As a result of this interchangeability.

Some of the general advice on designing server rendered applications are:

- don't access the DOM directly
- don't use `BrowserDomAdapter` it's only for internal use
- use a custom renderer

IDE/Editor

We have used Sublime and official Microsoft plugin for all our code here and we recommend it as an easy and light weight editor. Of course, heavyweight and extended IDEs are also available like Webstorm that one can use when leveraging extended features, but for most cases Sublime should suffice.

Linting

It is also important to conform to certain code quality standards, especially when designing applications for enterprises that are worked upon by a large group of developers.